

Location-Based Services

WHAT YOU WILL LEARN IN THIS CHAPTER

- Displaying Google Maps in your Android application
- Displaying zoom controls on the map
- Switching between the different map views
- Retrieving the address location touched on the map
- Performing geocoding and reverse geocoding
- Obtaining geographical data using GPS, Cell-ID, and Wi-Fi triangulation
- Monitoring for a location
- Building a location tracker application

CODE DOWNLOAD The wrox.com code downloads for this chapter are found at www.wrox.com/go/beginningandroidprog on the Download Code tab. The code is in the chapter 10 download and individually named according to the names throughout the chapter.

You have seen the explosive growth of mobile apps in recent years. One category of apps that is very popular is Location-Based Services, commonly known as LBS. LBS apps track your location, and might offer additional services such as locating amenities nearby, offering

WOW! eBook
www.wowebook.org

suggestions for route planning, and so on. Of course, one of the key ingredients in an LBS app is maps, which present a visual representation of your location.

This chapter shows you how to make use of Google Maps in your Android application, as well as how to manipulate the map view programmatically. In addition, you find out how to obtain your geographical location using the `LocationManager` class available in the Android SDK. This chapter ends with a project to build a location tracker application that you can install on an Android device and use to track the location of friends and relatives using SMS messaging.

DISPLAYING MAPS

Google Maps is one of the many applications bundled with the Android platform. In addition to simply using the Maps application, you can also embed it into your own applications and make it do some very cool things. This section describes how to use Google Maps in your Android applications and programmatically perform the following:

- Change the views of Google Maps.
- Obtain the latitude and longitude of locations in Google Maps.
- Perform geocoding and reverse geocoding (translating an address to latitude and longitude and vice versa).

Creating the Project

To get started, you need to first create an Android project so that you can display Google Maps in your activity:

1. Using Android Studio, create an Android project and name it LBS.
2. From the Create New Project Wizard, select Google Maps Activity as shown in Figure 10-1.

Obtaining the Maps API Key

Beginning with the Android SDK release v1.0, you need to apply for a free Google Maps API key before you can integrate Google Maps into your Android application. When you apply for the key, you must also agree to Google's terms of use, so be sure to read them carefully.

To get a Google Maps key, open the `google_maps_api.xml` file that was created in your LBS project. Within this file is a link to create a new Google Maps key. Simply copy and paste the link into your browser and follow the instructions. Make note of the key that Google gives you because you need it later in this project.

As usual, you need to create a subclass of the `AsyncTask` class to call the `WordDefinition()` method asynchronously:

```
private class AccessWebServiceTask extends AsyncTask<String, Void, String> {
    protected String doInBackground(String... urls) {
        return WordDefinition(urls[0]);
    }

    protected void onPostExecute(String result) {
        Toast.makeText(getApplicationContext(), result, Toast.LENGTH_LONG).show();
    }
}
```

Finally, you access the web service asynchronously using the `execute()` method:

```
///---access a Web Service using GET---
new AccessWebServiceTask().execute("apple");
```

CONSUMING JSON SERVICES

In the previous section, you learned how to consume XML web services by using HTTP to connect to the web server and then obtain the results in XML. You also learned how to use DOM to parse the result of the XML document. However, manipulating XML documents is a computationally expensive operation for mobile devices, for the following reasons:

- XML documents are lengthy. They use tags to embed information, and the size of an XML document can pretty quickly become large. A large XML document means that your device must use more bandwidth to download it, which translates into higher cost.
- XML documents are more difficult to process. As shown earlier, when using the `DocumentBuilderFactory`, you must use DOM to traverse the tree in order to locate the information you want. In addition, DOM itself has to build the entire document in memory as a tree structure before you can traverse it. This is both memory and CPU intensive.

A much more efficient way to represent information exists in the form of JSON (JavaScript Object Notation). JSON is a lightweight data-interchange format that is easy for humans to read and write. It is also easy for machines to parse and generate. The following lines of code show what a JSON message looks like:

```
[
  {
    "appleId": "1",
    "survId": "1",
    "location": "",
    "surveyDate": "2008-03-14",
    "surveyTime": "12:19:47",
    "inputUserId": "1",
    "inputTime": "2008-03-14 12:21:51",
    "modifyTime": "0000-00-00 00:00:00"
  }
]
```

WOW! eBook
www.wowebook.org

```
    },
    {
      "appleId": "2",
      "survId": "32",
      "location": "",
      "surveyDate": "2008-03-14",
      "surveyTime": "22:43:09",
      "inputUserId": "32",
      "inputTime": "2008-03-14 22:43:37",
      "modifyTime": "0000-00-00 00:00:00"
    },
    {
      "appleId": "3",
      "survId": "32",
      "location": "",
      "surveyDate": "2008-03-15",
      "surveyTime": "07:59:33",
      "inputUserId": "32",
      "inputTime": "2008-03-15 08:00:44",
      "modifyTime": "0000-00-00 00:00:00"
    },
    {
      "appleId": "4",
      "survId": "1",
      "location": "",
      "surveyDate": "2008-03-15",
      "surveyTime": "10:45:42",
      "inputUserId": "1",
      "inputTime": "2008-03-15 10:46:04",
      "modifyTime": "0000-00-00 00:00:00"
    },
    {
      "appleId": "5",
      "survId": "32",
      "location": "",
      "surveyDate": "2008-03-16",
      "surveyTime": "08:04:49",
      "inputUserId": "32",
      "inputTime": "2008-03-16 08:05:26",
      "modifyTime": "0000-00-00 00:00:00"
    },
    {
      "appleId": "6",
      "survId": "32",
      "location": "",
      "surveyDate": "2008-03-20",
      "surveyTime": "20:19:01",
      "inputUserId": "32",
      "inputTime": "2008-03-20 20:19:32",
      "modifyTime": "0000-00-00 00:00:00"
    }
  ]
}
```

WOW! eBook
www.wowebook.org

The preceding block of lines represents a set of data taken for a survey. Note that the information is represented as a collection of key/value pairs, and that each key/value pair is grouped into an ordered list of objects. Unlike XML, there are no lengthy tag names. Instead, there are only brackets and braces.

The following Try It Out demonstrates how to process JSON messages easily using the `JSONArray` and `JSONObject` classes available in the Android SDK.

TRY IT OUT Consuming JSON Services

1. Using Android Studio, create a new Android project and name it `JSON`.
2. Add the following bolded line to the `AndroidManifest.xml` file. Please be sure to replace any instance of `com.jfdimarzio` with the name of the package used in your application:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.jfdimarzio.json">
    <uses-permission android:name="android.permission.INTERNET"/>
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

3. Add the following lines of code in bold to the `MainActivity.java` file:

```
import android.os.AsyncTask;
import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;
import android.util.Log;
import android.widget.Toast;

import org.json.JSONArray;
import org.json.JSONObject;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.MalformedURLException;
import java.net.URL;
```

WOW! eBook
www.wowebook.org

```
public class MainActivity extends AppCompatActivity {
    public String readJSONFeed(String address) {
        URL url = null;
        try {
            url = new URL(address);
        } catch (MalformedURLException e) {
            e.printStackTrace();
        }
        StringBuilder stringBuilder = new StringBuilder();
        HttpURLConnection urlConnection = null;
        try {
            urlConnection = (HttpURLConnection) url.openConnection();
        } catch (IOException e) {
            e.printStackTrace();
        }
        try {
            InputStream content = new BufferedInputStream(
                urlConnection.getInputStream());
            BufferedReader reader = new BufferedReader(
                new InputStreamReader(content));
            String line;
            while ((line = reader.readLine()) != null) {
                stringBuilder.append(line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            urlConnection.disconnect();
        }
        return stringBuilder.toString();
    }
    private class ReadJSONFeedTask extends AsyncTask<String, Void, String> {
        protected String doInBackground(String... urls) {
            return readJSONFeed(urls[0]);
        }
        protected void onPostExecute(String result) {
            try {
                JSONArray jsonArray = new JSONArray(result);
                Log.i("JSON", "Number of surveys in feed: " +
                    jsonArray.length());
                //---print out the content of the json feed---
                for (int i = 0; i < jsonArray.length(); i++) {
                    JSONObject jsonObject = jsonArray.getJSONObject(i);
                    Toast.makeText(getApplicationContext(),
                        jsonObject.getString("appId") +
                            " - " + jsonObject.getString("inputTime"),
                            Toast.LENGTH_SHORT).show();
                }
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
}
```



```

    try {
        url = new URL(address);
    } catch (MalformedURLException e) {
        e.printStackTrace();
    };
    StringBuilder stringBuilder = new StringBuilder();
    HttpURLConnection urlConnection = null;
    try {
        urlConnection = (HttpURLConnection) url.openConnection();
    } catch (IOException e) {
        e.printStackTrace();
    }
    try {
        InputStream content = new BufferedInputStream(
            urlConnection.getInputStream());
        BufferedReader reader = new BufferedReader(
            new InputStreamReader(content));
        String line;
        while ((line = reader.readLine()) != null) {
            stringBuilder.append(line);
        }
    } catch (IOException e) {
        e.printStackTrace();
    } finally {
        urlConnection.disconnect();
    }
    return stringBuilder.toString();
}

private class ReadJSONFeedTask extends AsyncTask<String, Void, String> {
    protected String doInBackground(String... urls) {
        return readJSONFeed(urls[0]);
    }

    protected void onPostExecute(String result) {
        try {
            JSONArray jsonArray = new JSONArray(result);
            Log.i("JSON", "Number of surveys in feed: " +
                jsonArray.length());
            //---print out the content of the json feed---
            for (int i = 0; i < jsonArray.length(); i++) {
                JSONObject jsonObject = jsonArray.getJSONObject(i);
                Toast.makeText(getApplicationContext(),
                    jsonObject.getString("appId") +
                        " - " + jsonObject.getString("inputTime"),
                        Toast.LENGTH_SHORT).show();
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

WOW! eBook
www.wowebook.org

```

    }

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        new ReadJSONFeedTask().execute(
            "http://extjs.org.cn/extjs/examples/grid/survey.html");
    }
}

```

4. Press Shift+F9 to debug the application on the Android emulator. You see the Toast class appear a couple of times, displaying the information (see Figure 11-4).



FIGURE 11-4